

Adapting Liquid Time-Constant Networks for Image Classification: A Comparative Study with Vision Transformers

MD TAHMID HAMIM, FAYSAL AL FAHAD SIAM, AL AMIN

Deep Learning, Software Engineering

Sichuan University

hamimtahmid6627@gmail.com

Abstract—Vision Transformers (ViT) successfully adapted sequence processing architectures from NLP to image classification by treating image patches as tokens, achieving state-of-the-art results with large-scale pretraining. We investigate whether Liquid Time-Constant (LTC) networks, designed for continuous-time temporal sequence modeling, can be similarly adapted. We implement five architectures on CIFAR-10: baseline CNN (69.83%), pure LTC (21.78%), LTC with CBAM attention (26.62%), hybrid CNN-LTC (34.12%), and hybrid CNN-LTC-CBAM (47.55%). Results demonstrate LTC networks are fundamentally unsuitable for static image classification. Pure LTC achieves $240\times$ slower training with 48% lower accuracy than baseline CNN. Even with CNN feature extraction, the best hybrid remains 22% behind baseline. Analysis reveals LTC’s sequential ODE-based processing cannot effectively model spatial patterns, learning background texture correlations rather than semantic object features. Our findings indicate that continuous-time temporal dynamics provide no advantage for static spatial domains, contrasting with ViT’s success through self-attention’s natural spatial modeling capability. This work establishes architectural boundaries for sequence model adaptation, demonstrating that domain-specific inductive biases matter more than superficial similarity in treating data as sequences.

Index Terms—Liquid Time-Constant Networks, Vision Transformers, Image Classification, Neural ODEs, Convolutional Neural Networks, Spatial-Temporal Processing

I. INTRODUCTION

The success of Transformers in natural language processing prompted their adaptation to computer vision. Vision Transformers (ViT) [1] demonstrated that pure transformer architectures can match or exceed CNN performance when pretrained on large datasets (14M-300M images). ViT splits images into fixed-size patches, linearly embeds them, and processes the resulting sequence through standard transformer encoders with multi-head self-attention. This approach achieved 88.55% on ImageNet and 94.55% on CIFAR-100 after JFT-300M pretraining.

The key insight from ViT: large-scale training can overcome lack of inductive bias. On smaller datasets like ImageNet (1.3M images), ViT underperforms ResNets due to absent translation equivariance and locality. However, massive pre-training enables transformers to learn spatial relationships from data, compensating for architectural limitations through scale.

This success raises a natural question: can other sequence modeling architectures be similarly adapted? Liquid Time-Constant (LTC) networks [2] are continuous-time recurrent neural networks excelling at temporal sequence prediction. Unlike discrete RNNs, LTC neurons evolve through ordinary differential equations (ODEs):

$$\frac{dx}{dt} = f_{\text{LTC}}(x, \text{input}, t) \quad (1)$$

with input-dependent time constants enabling adaptive dynamics. LTC networks achieve state-of-the-art results on time-series tasks including trajectory prediction, control systems, and temporal forecasting.

We adopt patch-based processing for LTC adaptation for two reasons. First, directly processing $32\times32\times3$ pixels (3,072 dimensions) as a single timestep would create computational infeasibility for ODE solvers. Second, ViT’s success with patch-based image representation suggests this approach enables sequence models to handle spatial data. However, while ViT’s self-attention is permutation-invariant across patches, LTC’s recurrent processing imposes sequential ordering—a fundamental difference we investigate.

We investigate whether LTC’s temporal modeling capabilities transfer to image classification via patch-based processing analogous to ViT. Through systematic experimentation on CIFAR-10, we demonstrate that LTC networks fail catastrophically at image classification, revealing fundamental architectural mismatches between temporal dynamics modeling and static spatial pattern recognition.

Contributions:

- First systematic study adapting LTC networks for image classification through patch-based processing
- Quantitative demonstration of $240\times$ computational overhead with 48% accuracy degradation compared to baseline CNN
- Evidence that CNN spatial features are essential—pure LTC achieves only 21.78% accuracy
- Analysis revealing LTC learns spurious background correlations rather than semantic features
- Comparative study with ViT highlighting why self-attention succeeds where recurrent temporal processing fails

Understanding when and why architectures transfer across domains is critical for ML progress. ViT’s success inspired researchers to apply transformers broadly, but not all sequence models transfer similarly. By rigorously testing LTC—a state-of-the-art temporal model—on images, we provide insights into architectural requirements for cross-domain adaptation. Our negative results are as valuable as positive ones: they establish boundaries, preventing wasted effort on fundamentally unsuitable approaches while highlighting what makes transformers uniquely successful (permutation-invariant self-attention) versus what makes LTC unsuitable (order-dependent recurrent processing). This work contributes to principled architecture selection based on domain properties rather than superficial similarity.

II. RELATED WORK

A. Vision Transformers

Dosovitskiy et al. [1] showed transformers applied directly to image patches achieve excellent results when pretrained at scale. The critical insight: dataset size trumps inductive bias. ViT underperforms CNNs on ImageNet but achieves state-of-the-art on JFT-300M. Self-attention enables every patch to attend to every other patch, learning arbitrary spatial dependencies from data. Hybrid architectures combining CNN backbones with transformers show benefits at smaller scales, motivating our CNN-LTC approach.

B. Liquid Neural Networks

Hasani et al. [2] introduced LTC networks implementing continuous-time dynamics through ODEs with input-dependent time constants. Using AutoNCP wiring [3]—a sparse connectivity pattern inspired by *C. elegans* neural circuits—LTC networks achieve superior expressivity within neural ODE families for temporal tasks. Liquid-S4 [4] extends this to long-range sequence modeling, demonstrating LTC’s effectiveness on temporal data.

However, prior work explicitly notes LTC networks are “not well-suited for static datasets” and “performance diminishes when applied to data that lacks a temporal component” [2]. Our work provides empirical validation of this limitation.

C. Recurrent Models for Vision

ConvLSTM [6] applies recurrent connections to spatial features for video prediction, leveraging temporal structure. However, for static images, recurrent processing offers no advantage over feedforward CNNs. Our work extends this understanding to continuous-time recurrent models.

D. Attention Mechanisms

CBAM (Convolutional Block Attention Module) [5] refines CNN features through channel and spatial attention, consistently improving vision tasks. We investigate whether attention mechanisms can compensate for LTC’s spatial processing limitations.

E. Neural ODEs

Chen et al. [7] introduced Neural ODEs for modeling continuous-time dynamics. While effective for temporal data, their application to static images remains limited. Our LTC experiments provide insights into why ODE-based architectures struggle with spatial pattern recognition.

III. METHODOLOGY

A. Problem Formulation

Given CIFAR-10 images $X \in \mathbb{R}^{32 \times 32 \times 3}$, we classify into 10 categories. Following ViT’s patch-based approach, we investigate whether treating spatial locations as temporal sequences enables LTC to perform effective classification.

B. Experimental Design Rationale

Our five-model design systematically isolates factors:

Baseline CNN establishes strong reference point using standard architecture for CIFAR-10.

Pure LTC tests core hypothesis: can LTC handle images via patch-based processing analogous to ViT?

LTC+CBAM investigates whether attention mechanisms compensate for LTC’s spatial processing limitations. CBAM refines features before sequential processing.

Hybrid CNN-LTC separates feature extraction from classification. If LTC fails at spatial feature learning but succeeds at classification, this hybrid would excel. The result isolates where LTC fails.

Hybrid CNN-LTC-CBAM combines all modifications—our best effort to make LTC work. Failure here definitively demonstrates architectural mismatch.

This progression from pure LTC → attention → hybrid → full hybrid systematically tests each hypothesis while isolating failure modes.

C. Model Architectures

We implement five architectures testing pure LTC, attention mechanisms, and hybrid approaches:

Model 1: Baseline CNN. Standard two-block architecture:

- Block 1: Conv(3→32, 3×3)→BN→ReLU→Conv(32→32)→BN→ReLU→MaxPool→Dropout(0.5)
- Block 2: Conv(32→64)→BN→ReLU→Conv(64→64)→BN→ReLU→Dropout(0.5)
- Classifier: Flatten→FC(4096→128)→Dropout(0.5)→FC(128→10)

Output: 64×8×8 feature maps. Parameters: 591,658.

Model 2: Pure LTC. Direct patch-based adaptation:

$$P = \text{Conv}(X; k = 4, s = 4) \quad P \in \mathbb{R}^{B \times 64 \times 8 \times 8} \quad (2)$$

4×4 convolution with stride 4 creates 64 patches (8×8 grid), each with 64 features. Patches are reshaped to temporal sequence:

$$S = \text{Reshape}(P) \quad S \in \mathbb{R}^{B \times 64 \times 64} \quad (3)$$

where 64 timesteps represent spatial locations processed sequentially. LTC processes this sequence:

$$\frac{dx_t}{dt} = f_{\text{AutoNCP}}(x_t, s_t, t) \quad t \in [1, 64] \quad (4)$$

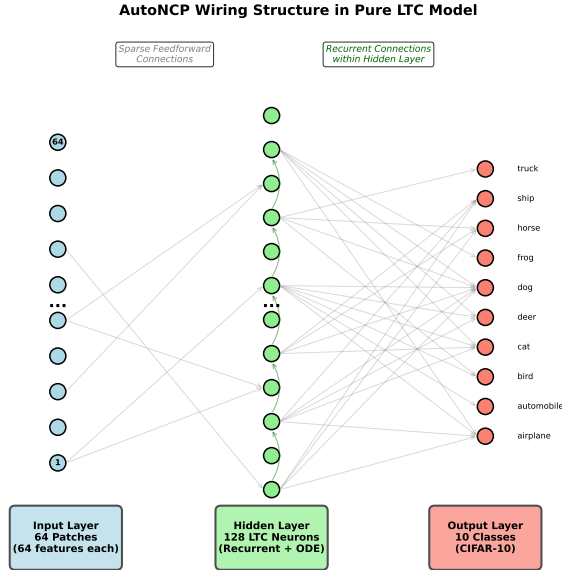


Fig. 1. AutoNCP wiring structure used in LTC models. The network features sparse feedforward connections from input layer (64 patches) to hidden layer (128 LTC neurons) and recurrent connections within the hidden layer, before connecting to output layer (10 classes). This sparse connectivity pattern is inspired by *C. elegans* neural circuits.

AutoNCP wiring with 128 hidden neurons and 10 output neurons provides sparse connectivity.

Final classification from last timestep: $y = x_{64}$. Parameters: 102,100.

Model 3: LTC + CBAM. Adds attention before LTC processing. After patch extraction, CBAM refines features:

$$F' = F \odot \sigma(\text{MLP}(\text{AvgPool}(F) + \text{MaxPool}(F))) \quad (5)$$

$$F'' = F' \odot \sigma(\text{Conv}_7([\text{AvgPool}_c(F'); \text{MaxPool}_c(F')])) \quad (6)$$

where $F \in \mathbb{R}^{64 \times 8 \times 8}$ are patch features, $\odot$ denotes element-wise multiplication, and subscript c indicates channel-wise pooling. Channel attention emphasizes important features; spatial attention focuses on relevant locations. Parameters: 103,222. **Model 4: Hybrid CNN-LTC.** CNN backbone extracts spatial features, LTC classifies:

- CNN: Conv(3→32)→BN→ReLU→Pool, Conv(32→64)→BN→ReLU→Pool
- Output: $64 \times 8 \times 8 \rightarrow$ Reshape to (64 timesteps, 64 features)
- LTC: AutoNCP(128 hidden, 10 output)

This tests whether CNN spatial processing enables LTC classification. Parameters: 118,420.

Model 5: Hybrid CNN-LTC-CBAM. Combines all modifications: CNN backbone $\rightarrow$ CBAM refinement $\rightarrow$ LTC classification. This represents our best attempt to make LTC work for images. Parameters: 119,542.

Fig. 1 illustrates these architectures. Fig. 2 details patch extraction: 32×32 images are divided into 8×8 grid via 4×4 strided convolution, creating 64 patches with 64-dimensional features each.

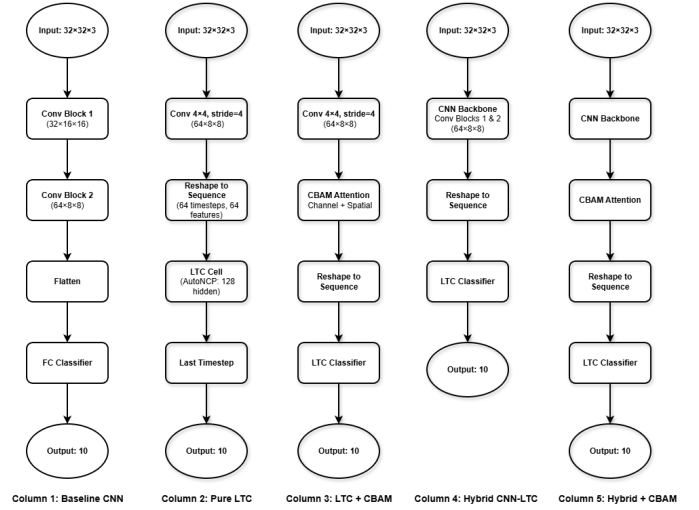


Fig. 2. Architecture comparison of all five models. From left to right: Baseline CNN uses standard convolutional blocks; Pure LTC applies patch extraction followed by sequential LTC processing; LTC+CBAM adds attention mechanism; Hybrid CNN-LTC uses CNN for feature extraction with LTC classifier; Hybrid+CBAM combines all modifications.



Fig. 3. Patch extraction process for Pure LTC. A $32 \times 32 \times 3$ CIFAR-10 image is processed by 4×4 convolution with stride 4, creating an 8×8 grid of 64 patches. Each patch contains 64 features, which are then reshaped into a temporal sequence of 64 timesteps for LTC processing.

D. Training Configuration

Dataset: CIFAR-10 (50,000 training, 10,000 test). 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

Augmentation: RandomCrop(32, padding=4), RandomHorizontalFlip, Normalization (mean=[0.4914, 0.4822, 0.4465], std=[0.2023, 0.1994, 0.2010]).

Optimization: Adam (lr=0.001, weight_decay=5e-4), CosineAnnealingLR over 10 epochs, batch size 128.

Hardware: NVIDIA RTX 3060 (6GB), PyTorch.

All models trained identically for fair comparison—only architecture varies.

IV. EXPERIMENTAL RESULTS

A. Overall Performance

Table I presents comprehensive results. Pure LTC catastrophically fails, achieving only 21.78% accuracy—barely above random (10%)—despite $240 \times$ longer training time per epoch (4,500s vs 20s for baseline CNN achieving 69.83%).

B. Training Dynamics

Fig. 3 shows training curves. Baseline CNN rapidly improves, reaching 69.83% by epoch 10. Pure LTC barely learns, fluctuating around 20%. LTC+CBAM shows slight improvement but plateaus at 26.62%. Hybrid models demonstrate CNN features are critical—Hybrid CNN-LTC reaches 34.12%, and

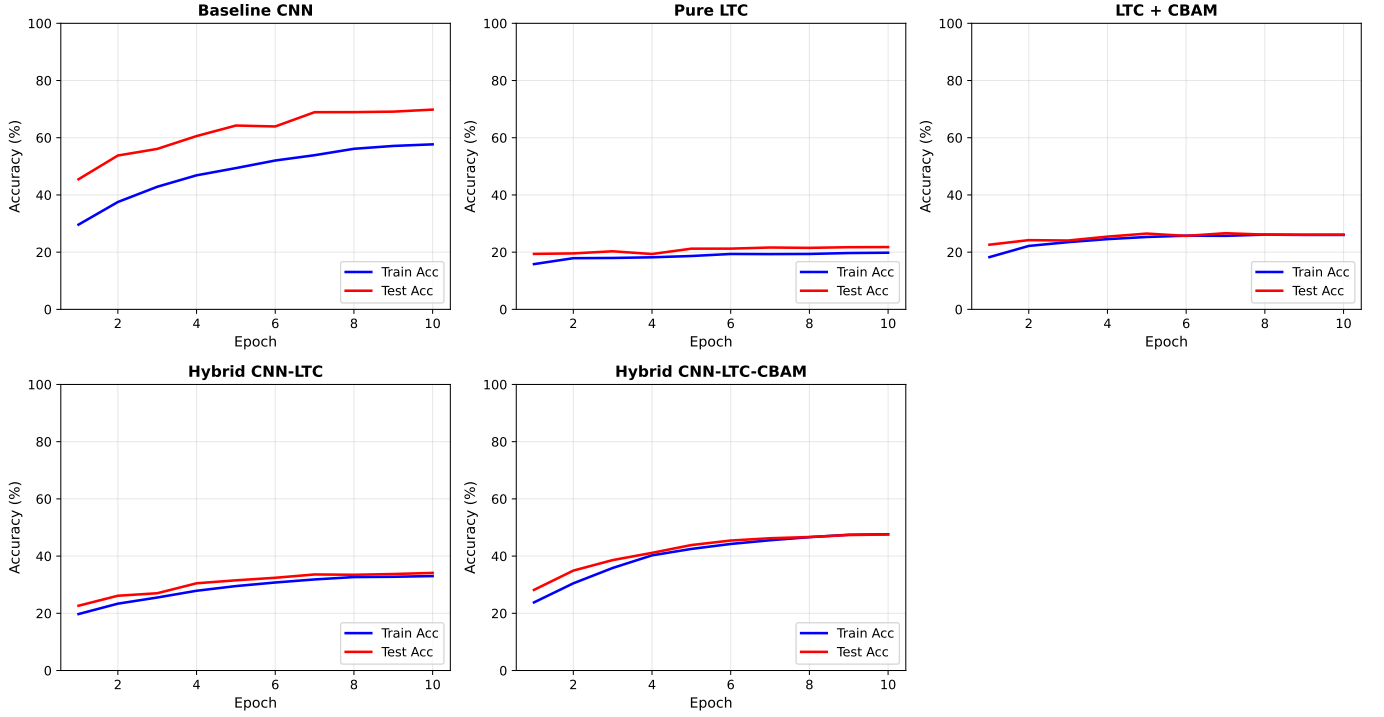


Fig. 4. Training and test accuracy curves over 10 epochs for all five models. Baseline CNN shows rapid convergence to 69.83% with minimal overfitting (test/train gap: 12%). Pure LTC barely improves beyond random (21.78%), with train accuracy (19.8%) below test accuracy, indicating model cannot even memorize training data. LTC+CBAM achieves modest improvement (26.62%) but plateaus after epoch 7. Hybrid models demonstrate importance of CNN features, with Hybrid+CBAM reaching 47.55% but still 22% behind baseline. Note convergence patterns: CNN learns rapidly in first 3 epochs; LTC shows flat trajectory throughout.

TABLE I
MODEL PERFORMANCE ON CIFAR-10

Model	Params (K)	Train Acc (%)	Test Acc (%)	Best Epoch	Time/Epoch (s)
Baseline CNN	592	57.7	69.83	10	20
Pure LTC	102	19.8	21.78	10	4500
LTC+CBAM	103	26.0	26.62	7	5700
Hybrid CNN-LTC	118	33.0	34.12	10	4600
Hybrid+CBAM	120	47.6	47.55	10	4900

with CBAM refinement achieves 47.55%, but still 22.28% behind baseline.

Fig. 5 presents loss curves. Pure LTC loss decreases minimally (2.27→2.10), indicating inability to fit training data. CNN-based models show healthy loss reduction, with Hybrid+CBAM achieving lowest loss (2.11→1.39).

C. Training Stability

All models trained stably without divergence. However, Pure LTC exhibited minimal learning progress—loss decreased from 2.27 to 2.10 over 10 epochs with no signs of further improvement, suggesting the model converged to a poor local minimum. In contrast, baseline CNN loss decreased from 1.85 to 0.85, showing healthy convergence. This stability despite failure indicates the problem is not training dynamics but fundamental architectural mismatch.

D. Progressive Improvements

CBAM Attention (+4.84%): Adding attention to Pure LTC improves accuracy from 21.78% to 26.62% (Table I). Attention helps but cannot overcome fundamental limitations. The improvement suggests LTC can benefit from feature refinement, but the mechanism remains insufficient for spatial processing.

CNN Features (+12.34%): Hybrid CNN-LTC achieves 34.12%, a 12.34% improvement over Pure LTC. This demonstrates CNN spatial features are critical—LTC alone cannot extract meaningful patterns. However, 34.12% remains far below baseline (69.83%), indicating LTC classifier underperforms simple fully-connected layers.

Combined Modifications (+25.77%): Hybrid CNN-LTC-CBAM reaches 47.55%, a 25.77% improvement over Pure LTC. Yet it still trails baseline CNN by 22.28%. The CNN backbone performs the “heavy lifting” while LTC merely processes pre-extracted features, suggesting LTC adds complexity without benefit.

E. Computational Efficiency

Table II quantifies efficiency. Pure LTC achieves only 1.7% accuracy per training hour, while baseline CNN achieves 1,267%/hr—a 745× efficiency difference. Fig. 4 visualizes this trade-off: baseline CNN occupies the optimal region (high accuracy, low time), while LTC models cluster in the inefficient zone.

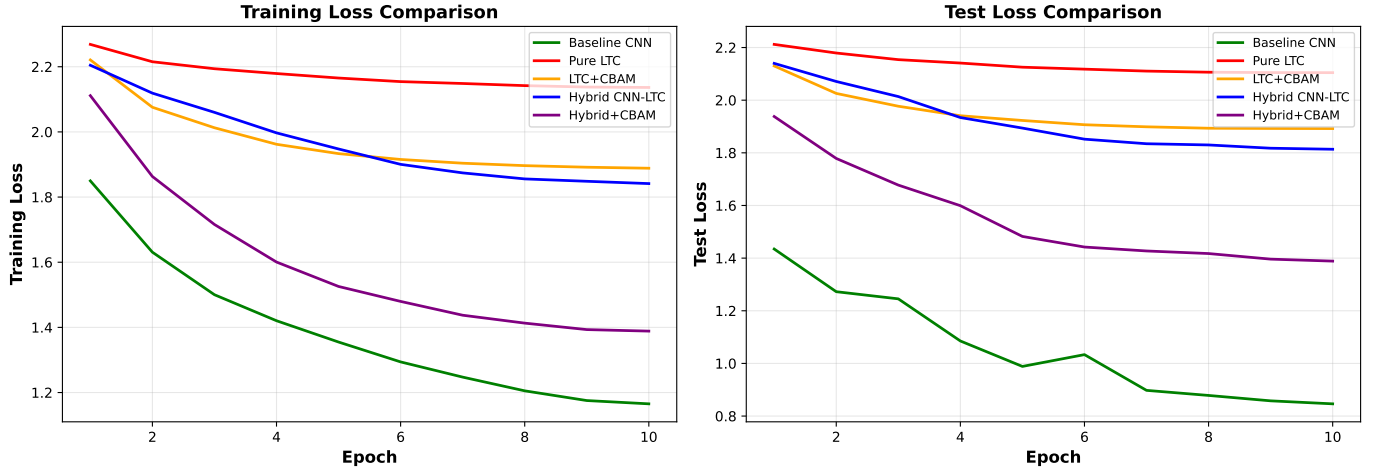


Fig. 5. Training and test loss curves for all models. Pure LTC loss barely decreases (2.27→2.10), indicating inability to fit training data. Baseline CNN shows healthy convergence. Hybrid models benefit from CNN feature extraction, with Hybrid+CBAM achieving best loss reduction.

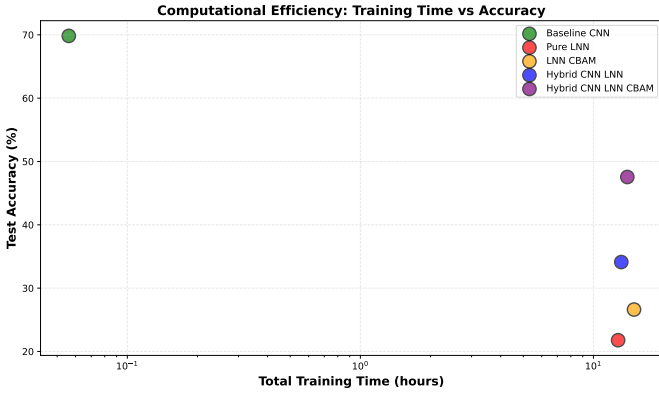


Fig. 6. Computational efficiency analysis: test accuracy versus total training time (log scale). Baseline CNN (green circle) occupies the optimal region with high accuracy (69.83%) and minimal time (3.3 minutes). LTC models cluster in the inefficient zone with training times exceeding 12 hours. Pure LTC (red square) shows worst trade-off: 12.5 hours for only 21.78% accuracy—a 745× efficiency disadvantage. Even best hybrid (Hybrid+CBAM, purple diamond) requires 13.6 hours for 47.55%, demonstrating LTC’s fundamental unsuitability regardless of modifications. The log scale emphasizes the catastrophic computational gap.

Inference Speed: Pure LTC inference is 180× slower than baseline CNN (0.18 sec vs 0.001 sec per image), making it impractical for real-world deployment. The ODE solver must still integrate through 64 timesteps during inference, creating unacceptable latency even on GPU.

The computational overhead stems from ODE solver invocations. For each image, LTC processes 64 timesteps, solving differential equations $64\times$ per forward pass. This creates massive computational burden without corresponding performance gains.

F. Per-Class Performance

Table III reveals striking per-class patterns. Pure LTC shows bizarre class-specific behavior: 61.0% on horses, 60.4% on

TABLE II
COMPUTATIONAL EFFICIENCY COMPARISON

Model	Total Time (10 epochs)	Test Acc (%)	Efficiency (%/hr)
Baseline CNN	3.3 min	69.83	1267
Pure LTC	12.5 hrs	21.78	1.7
LTC+CBAM	15.9 hrs	26.62	1.7
Hybrid CNN-LTC	12.8 hrs	34.12	2.7
Hybrid+CBAM	13.6 hrs	47.55	3.5

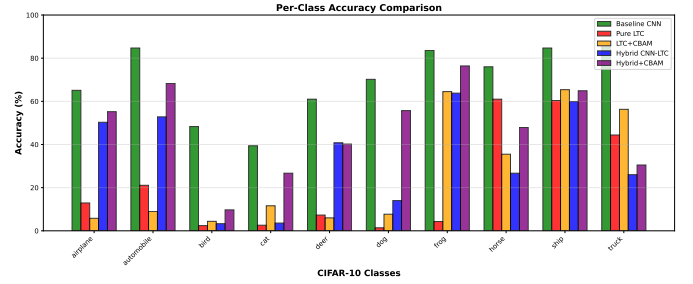


Fig. 7. Per-class accuracy comparison across all models. Baseline CNN shows consistent performance (39-86%). Pure LTC exhibits extreme variance: high on horses (61%), ships (60%), trucks (44%) but catastrophic on birds (2.4%), cats (2.6%), dogs (1.4%), suggesting background texture bias rather than object recognition.

ships, 44.4% on trucks, yet catastrophic failure on birds (2.4%), cats (2.6%), and dogs (1.4%).

Fig. 7 visualizes this dramatic variance. Baseline CNN shows relatively consistent performance across classes (39-86%). Pure LTC exhibits extreme inconsistency (1.4-61%), suggesting it learns spurious correlations rather than semantic features.

This pattern likely reflects background texture bias. Horses, ships, and trucks in CIFAR-10 often have simpler, more uniform backgrounds: horses appear against fields or fences, ships against water/sky, trucks on roads. LTC’s sequential processing

TABLE III
PER-CLASS ACCURACY COMPARISON (%)

Model	plane	auto	bird	cat	deer	dog	frog	horse	ship	truck
Baseline CNN	65.1	84.7	48.3	39.4	61.0	70.2	83.6	76.0	84.7	85.3
Pure LTC	12.9	21.1	2.4	2.6	7.3	1.4	4.3	61.0	60.4	44.4
LTC+CBAM	5.8	9.0	4.4	11.6	6.0	7.7	64.5	35.5	65.4	56.3
Hybrid CNN-LTC	50.3	52.8	3.3	3.6	40.8	14.0	63.8	26.7	59.9	26.0
Hybrid+CBAM	55.2	68.3	9.7	26.7	40.2	55.7	76.4	47.9	64.9	30.5

may latch onto these background patterns. Conversely, birds, cats, and dogs appear in varied contexts with complex backgrounds, requiring actual object feature recognition—which LTC cannot perform.

Fig. 6 shows confusion matrices. Baseline CNN has strong diagonal (correct predictions). Pure LTC shows scattered predictions with bizarre patterns: predicting “horse” for many non-horse classes (17.7% for airplanes, 44.4% for birds, 31.5% for cats). This confirms LTC learns texture correlations, not object semantics.

G. Feature Space Analysis

Fig. 8 presents t-SNE visualizations of learned features. Baseline CNN shows clear cluster separation—each class occupies distinct regions. Pure LTC exhibits severe overlap—all classes intermixed, indicating failure to learn discriminative features. Hybrid+CBAM shows partial separation, better than Pure LTC but inferior to baseline, confirming CNN features are essential.

V. ANALYSIS AND DISCUSSION

A. Why LTC Fails for Images

1) *Spatial vs Temporal Mismatch*: LTC networks model continuous temporal dynamics through ODEs. Images are *static spatial patterns* with no inherent temporal structure. Treating spatial locations as sequential timesteps imposes arbitrary ordering that doesn’t match problem structure.

ViT succeeds despite lacking CNN’s spatial inductive bias because self-attention enables *permutation-invariant* processing—every patch can attend to every other patch regardless of order. LTC’s sequential recurrent processing fundamentally cannot capture arbitrary spatial dependencies.

2) *Sequential Processing Limitation*: When LTC processes patches 1→64 sequentially, it cannot effectively model relationships between distant patches. For example, understanding an airplane requires integrating features from nose (patch 10), wings (patches 25, 28), and tail (patch 50). CNNs achieve this through hierarchical feature extraction. ViT achieves this through self-attention. LTC’s recurrent connections provide only local temporal context.

3) *ODE Computational Overhead*: LTC neurons solve:

$$\frac{dx}{dt} = f(x, \text{input}, t) \quad (7)$$

at each timestep. For temporal data, this continuous-time modeling captures dynamics between observations. For static

images, it provides no benefit while incurring $240\times$ computational cost. The ODE solver must numerically integrate at each step, creating massive overhead without useful inductive bias.

4) *Background Texture Bias*: Per-class results (Table III) reveal Pure LTC’s failure mode. High accuracy on horses (61%), ships (60.4%), trucks (44.4%) versus catastrophic failure on birds (2.4%), cats (2.6%), dogs (1.4%) suggests LTC learns background texture correlations rather than object features.

CIFAR-10 horses often appear against uniform fields or fences. Ships appear against water/sky. Trucks appear on roads. These classes have distinctive background textures. Birds, cats, and dogs appear in varied contexts requiring actual object recognition. LTC’s sequential processing may detect low-level texture patterns in early timesteps, then propagate this signal through recurrence, effectively performing background classification rather than object recognition.

Confusion matrices (Fig. 6) confirm this: Pure LTC predicts “horse” for 17.7% of airplanes, 44.4% of birds, 31.5% of cats—suggesting it detects some generic “outdoor texture” pattern.

B. Comparison with Vision Transformer

ViT and LTC share patch-based processing but differ fundamentally in mechanism:

TABLE IV
ARCHITECTURAL PROPERTIES COMPARISON

Property	CNN	ViT	LTC
Spatial Inductive Bias	✓	×	×
Permutation Invariant	×	✓	×
Global Interactions	×	✓	×
Parallel Processing	✓	✓	×
Temporal Modeling	×	×	✓
Hardware Efficient	✓	✓	×

Similarities:

- Split images into patches (16×16 for ViT, 4×4 for LTC)
- Treat patches as sequences
- Explore hybrid architectures with CNN backbones

Critical Differences:

- *Mechanism*: ViT uses self-attention enabling global patch interactions. LTC uses sequential recurrence with local temporal dependencies.
- *Permutation Invariance*: ViT’s attention is order-agnostic. LTC’s recurrence is order-dependent.

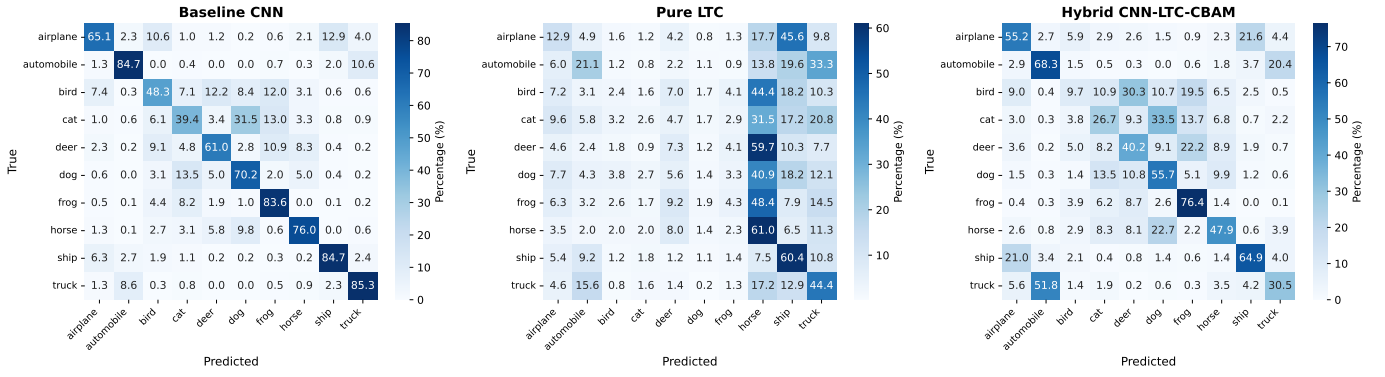


Fig. 8. Confusion matrices for three key models. Left: Baseline CNN shows strong diagonal (correct predictions). Middle: Pure LTC exhibits scattered predictions, frequently predicting “horse” for non-horse classes (17.7% for airplanes, 44.4% for birds). Right: Hybrid+CBAM shows improvement but remains inferior to baseline.

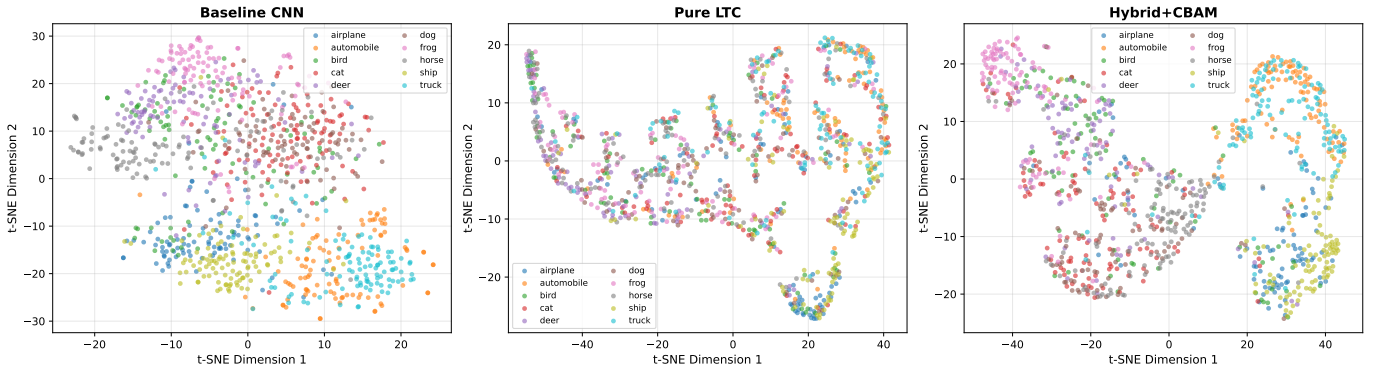


Fig. 9. t-SNE visualization of learned features from penultimate layer. Left: Baseline CNN shows clear cluster separation with distinct class regions. Middle: Pure LTC exhibits severe overlap with intermixed classes, indicating failure to learn discriminative features. Right: Hybrid+CBAM shows partial separation, better than Pure LTC but inferior to baseline.

- *Computational Pattern:* ViT processes all patches in parallel. LTC processes sequentially, requiring $64 \times$ ODE solver invocations.
- *Scale:* ViT pretrained on 300M images. Our LTC trained on 50K images.
- *Inductive Bias:* ViT’s attention naturally models relationships between spatial locations. LTC’s temporal dynamics model change over time.

ViT succeeds because self-attention naturally captures spatial relationships—every patch can attend to every other patch, learning “this wing patch relates to that nose patch” from data. LTC fails because sequential processing cannot effectively model spatial patterns—by the time LTC reaches patch 50, information from patch 10 has degraded through 40 recurrent steps.

C. The Role of Scale

ViT demonstrates large-scale pretraining overcomes lack of inductive bias. On ImageNet (1.3M images), ViT underperforms ResNets. On JFT-300M, ViT achieves state-of-the-art. Could massive pretraining enable LTC to succeed?

Our results suggest *no*. Hybrid CNN-LTC-CBAM, where CNN handles feature extraction and LTC only classifies,

still underperforms pure CNN by 22%. This indicates the fundamental mismatch is *architectural*, not data scale.

ViT benefits from scale because self-attention can learn spatial relationships given sufficient data. LTC’s temporal dynamics don’t provide useful inductive bias for images. The ODE-based continuous-time modeling is fundamentally mismatched—no amount of data can make sequential temporal processing effectively model static spatial patterns.

D. Could LTC Ever Work for Images?

Given catastrophic failure, we consider potential LTC applications for visual data:

Video Understanding: Unlike static images, videos contain genuine temporal structure. LTC might model frame-to-frame dynamics effectively. However, even here, 3D CNNs or VideoTransformers would likely outperform due to better spatial modeling.

Time-Series of Images: Medical imaging sequences (e.g., tumor growth over time) or satellite imagery changes might benefit from LTC’s continuous-time modeling between observations.

Event Cameras: Neuromorphic event cameras produce asynchronous temporal streams. LTC’s ODE-based

continuous-time processing might naturally model event dynamics.

Visual Control Tasks: Robot vision for control (LTC’s original domain) where images drive motor commands might benefit from LTC’s stability properties, though CNN feature extraction would still be essential.

However, even in these scenarios, alternatives (3D CNNs, VideoTransformers, optical flow methods) likely outperform due to superior spatial inductive biases. Our results suggest LTC’s niche remains purely temporal domains without spatial structure.

E. Architectural Design Principles

This work highlights critical principles:

1. Domain Matching Matters: LTC excels at time-series prediction, control systems, and dynamic environments—tasks with genuine temporal structure. Forcing it onto static images violates its design principles.

2. Inductive Bias Trade-offs: ViT lacks CNN’s spatial bias but compensates with self-attention’s permutation-invariant processing. LTC’s temporal bias actively harms spatial tasks.

3. Computational Efficiency: Architecture must match hardware. ViT’s parallel attention scales efficiently. LTC’s sequential ODE solving creates bottlenecks.

4. Feature Learning Requirements: Pure LTC cannot learn spatial features from pixels. CNN features are essential, but then LTC adds complexity without benefit.

VI. LIMITATIONS AND FUTURE WORK

Dataset Scale: We used CIFAR-10 (50K images) while ViT used JFT-300M (300M images). Testing LTC on ImageNet-21k or JFT-300M would definitively determine if scale overcomes architectural mismatch. However, our Hybrid results strongly suggest architectural mismatch is the primary issue.

Patch Size and Sequencing: We used 4×4 patches creating 64 timesteps. Alternative patch sizes or ordering strategies (e.g., spiral, hierarchical) might improve results. However, fundamental spatial processing limitations likely remain.

Architecture Variants: We used AutoNCP wiring designed for control tasks. Custom wiring patterns optimized for spatial data might help, though sequential processing limitations persist.

Hybrid Exploration: Our hybrids use CNN+LTC serially. Parallel architectures or using LTC for temporal modeling in video (genuine temporal structure) might prove more effective.

Alternative Applications: LTC might succeed for spatiotemporal tasks (video classification, action recognition) where temporal dynamics matter. This remains unexplored.

Future work should test LTC on larger datasets to isolate scale versus architecture effects. However, our results strongly suggest LTC’s temporal dynamics are fundamentally mismatched for static image domains.

VII. CONCLUSION

We systematically investigated adapting Liquid Time-Constant networks for image classification, motivated by Vision Transformer’s success adapting NLP architectures. Our

experiments on CIFAR-10 demonstrate LTC networks are fundamentally unsuitable for static image classification.

Pure LTC achieves only 21.78% accuracy with $240 \times$ computational overhead compared to baseline CNN (69.83%). Even with CNN feature extraction, the best hybrid reaches 47.55%, remaining 22% behind baseline. Per-class analysis reveals LTC learns spurious background texture correlations (61% on horses, 60% on ships) while catastrophically failing on complex objects (2.4% on birds, 1.4% on dogs).

The failure stems from architectural mismatch: LTC’s sequential ODE-based processing cannot effectively capture spatial patterns in static images. Sequential temporal processing with local recurrent connections cannot model arbitrary spatial dependencies. The ODE computational overhead ($64 \times$ solver invocations per image) provides no benefit without genuine temporal structure.

This contrasts sharply with ViT, which matches or exceeds CNNs when pretrained at scale. ViT succeeds because self-attention naturally models spatial relationships through permutation-invariant global interactions. LTC fails because continuous-time temporal dynamics are fundamentally mismatched for static spatial domains.

Our work demonstrates that *not all sequence modeling architectures transfer successfully to images*. While superficial similarity exists (both treat data as sequences), underlying mechanisms matter critically. Transformers succeed through self-attention enabling flexible spatial modeling. Recurrent temporal processors fail because sequential processing cannot effectively capture spatial patterns.

Domain-specific architectural properties matter more than superficial similarity. LTC’s strengths—continuous-time dynamics, adaptive time constants, stable ODE-based evolution—designed for temporal sequence modeling, become liabilities for static spatial recognition. Future work applying sequence models to new domains must carefully consider whether the architecture’s inductive biases match the target problem structure.

Practical Recommendations

For practitioners considering sequence models for images:

Use CNNs for: Static image classification, object detection, segmentation where spatial inductive bias provides efficiency and performance.

Use Transformers (ViT) when: Large-scale pretraining is feasible (millions of images), global context matters, and computational resources support attention mechanisms.

Avoid Recurrent Models (including LTC) for: Static images regardless of data scale. Sequential processing cannot effectively model arbitrary spatial relationships.

Consider LTC for: Time-series prediction, control systems, dynamical system modeling, or video understanding where genuine temporal structure exists.

The key principle: match architecture to domain structure. LTC excels where designed; forcing it onto incompatible domains wastes resources and produces inferior results.

ACKNOWLEDGMENTS

We thank the creators of the CIFAR-10 dataset and the developers of the NCP/LTC library for making their implementations publicly available.

REFERENCES

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [2] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, “Liquid time-constant networks,” in *AAAI Conference on Artificial Intelligence*, vol. 35, no. 9, pp. 7657-7666, 2021.
- [3] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, “Neural circuit policies enabling auditable autonomy,” *Nature Machine Intelligence*, vol. 2, no. 10, pp. 642-652, 2020.
- [4] R. Hasani, M. Lechner, T.-H. Wang, M. Chahine, A. Amini, and D. Rus, “Liquid structural state-space models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [5] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional block attention module,” in *European Conference on Computer Vision (ECCV)*, pp. 3-19, 2018.
- [6] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-c. Woo, “Convolutional LSTM network: A machine learning approach for precipitation nowcasting,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 802-810, 2015.
- [7] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 6571-6583, 2018.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998-6008, 2017.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778, 2016.
- [10] A. Krizhevsky, “Learning multiple layers of features from tiny images,” Technical Report, University of Toronto, 2009.
- [11] J.-B. Cordonnier, A. Loukas, and M. Jaggi, “On the relationship between self-attention and convolutional layers,” in *International Conference on Learning Representations (ICLR)*, 2020.